Angular Components

Web Development — Lecture 5

Seitbekov Sanzhar

KAZAKH-BRITISH TECHNICAL UNIVERSITY

KBTU · 2025 / 2026

Lecture Plan

PART 1 · 50 MIN

Component Deep Dive

→ Component anatomy recap

→ Component properties & metadata

→ The four types of Data Binding

→ Templates — expressions & directives

→ Styles & View Encapsulation

PART 2 · 50 MIN

Lifecycle & Live Coding

→ Component Lifecycle Hooks

→ ngOnInit, ngOnChanges, ngOnDestroy

→ Parent ↔ Child communication

→ Live coding: Multi-component Task App

→ Practice exercises

Quick Recap: What is a Component?

```ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-student-card',
  templateUrl: './student-card.html',
  styleUrl: './student-card.css'
})
export class StudentCardComponent {
  // Properties (data)
  name: string = 'Alice';
  grade: number = 95;

  // Methods (behavior)
  isPassing(): boolean {
    return this.grade >= 60;
  }
}
```

📄

Template

The HTML that defines what renders on screen

⚙️

Class

TypeScript code — data & logic

🎨

Styles

CSS scoped to this component only

🏷️

Decorator

@Component — tells Angular this is a component

01

Component Properties

Metadata, inputs, outputs — what makes a component tick

@Component Decorator Metadata

Property	Description
selector	Custom HTML tag name
templateUrl	Path to external HTML file
template	Inline HTML (short templates)
styleUrl	Path to external CSS file
styles	Inline CSS array
imports	Other components/modules used
standalone	Default: `true` in Angular 17+

```ts
@Component({
  selector: 'app-student-card',

  // Option A — external file
  templateUrl: './student-card.html',

  // Option B — inline template
  template: `<h1>Hello {{ name }}</h1>`,

  // Option A — external file
  styleUrl: './student-card.css',

  // Option B — inline styles
  styles: `h1 { color: #dd0031; }`,

  imports: [FormsModule, CommonModule],
  standalone: true
})
```

Rule: Use `template`/`styles` for tiny components, external files for anything bigger.

Class Properties

```ts
export class StudentCardComponent {
  // ① Regular properties
  name: string = 'Alice';
  grade: number = 95;
  isActive: boolean = true;
  tags: string[] = ['A+', 'honor'];

  // ② Signal-based (reactive)
  name = signal('Alice');
  grade = signal(95);

  // ③ Computed (derived values)
  isPassing = computed(() =>
    this.grade() >= 60
  );

  // ④ Methods
  promote() {
    this.grade.update(g => g + 5);
  }
}
```

REGULAR PROPERTIES

Standard TypeScript class fields. Angular detects changes automatically with Zone.js or Signals.

SIGNAL()

Reactive primitives — Angular knows exactly *when* and *what* changed. No full tree check needed.

COMPUTED()

Derived values that automatically update when their dependencies change. Like a spreadsheet formula.

Generating Components with CLI

```bash
# Full command
ng generate component student-card

# Shorthand
ng g c student-card

# Creates 4 files:
# src/app/student-card/
#   student-card.ts        ← component class
#   student-card.html      ← template
#   student-card.css       ← styles
#   student-card.spec.ts   ← unit test
```

What gets generated

→ Standalone component by default (Angular 17+)

→ Auto-imports added where component is used

→ Spec file for unit testing

→ Proper file naming convention (kebab-case)

Pro tip: Always use the CLI to generate components — it handles naming conventions, creates all necessary files, and keeps your project structure consistent.

02

Data Binding

How data flows between the component class and the DOM

The Four Types of Data Binding

`{{ }}` **Interpolation** Component → View
Embeds expressions as text in the template

`[prop]` **Property Binding** Component → View
Sets an element property to a component expression

`(event)` **Event Binding** View → Component
Listens to DOM events and calls component methods

`[(ng)]` **Two-way Binding** Component ↔ View
Combines property & event binding — syncs input fields with data

> **Memory trick:** {{ }} = string output, [ ] = into DOM, ( ) = out from DOM, [( )] = banana in a box 🍌📦

Interpolation {{ }}

```html
<!-- Simple property -->
<h1>Hello, {{ studentName }}!</h1>

<!-- Arithmetic -->
<p>Total: {{ price * quantity }}</p>

<!-- String concatenation -->
<p>{{ firstName + ' ' + lastName }}</p>

<!-- Method call -->
<p>Status: {{ isPassing() ? 'Pass' : 'Fail' }}</p>

<!-- Safe navigation -->
<p>City: {{ student?.address?.city }}</p>

<!-- Nullish coalescing -->
<p>{{ student?.name ?? 'Unknown' }}</p>
```

What you CAN use

→ Property access: user.name

→ Method calls: getLabel()

→ Operators: +, -, *, /, %

→ Ternary: a ? b : c

→ Safe nav: user?.email

→ Nullish: name ?? 'default'

❌ **NOT allowed:** assignments (=), new, increment (++), chaining (;). Keep expressions simple and side-effect free.

Property Binding [ ]

```html
<!-- Image src from component -->
<img [src]="student.photoUrl"
     [alt]="student.name">

<!-- Disable button based on condition -->
<button [disabled]="!isFormValid">
  Submit
</button>

<!-- Class binding -->
<div [class.active]="isSelected">Card</div>

<!-- Style binding -->
<div [style.background-color]="bgColor">
  Styled
</div>

<!-- Binding to an input of child component -->
<app-student-card
  [name]="student.name"
  [grade]="student.grade"/>
```

String vs Property Binding

```html
<!-- Hardcoded string (no binding) -->
<img src="avatar.png">

<!-- Dynamic (property binding) -->
<img [src]="student.photoUrl">
```

Without [ ] — Angular treats the value as a *string literal*. With [ ] — Angular evaluates it as a *TypeScript expression*.

Key insight: Property binding flows *into* the DOM. You're setting a DOM element's property to the result of an expression.

Event Binding ( )

```html
<!-- Click event -->
<button (click)="onSave()">Save</button>

<!-- Pass the $event object -->
<input (input)="onInput($event)">

<!-- Keyboard shortcut -->
<input (keyup.enter)="onSubmit()">

<!-- Mouse events -->
<div (mouseenter)="highlight()"
     (mouseleave)="unhighlight()">
  Hover me
</div>
```

```ts
export class MyComponent {
  onSave() {
    console.log('Saved!');
  }

  onInput(event: Event) {
    const input = event.target as
HTMLInputElement;
    console.log(input.value);
  }

  onSubmit() {
    console.log('Enter pressed!');
  }
}
```

$event is the DOM event object. Cast it to the correct type to access properties like `input.value`.

Two-way Binding [( )]

```ts
import { FormsModule } from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <input [(ngModel)]="searchTerm"
           placeholder="Search...">
    <p>Searching for: {{ searchTerm }}</p>
  `
})
export class SearchComponent {
  searchTerm: string = '';
}
```

How it actually works

Two-way binding is *syntactic sugar* for combining both directions:

```html
<!-- This is equivalent to: -->
<input
  [ngModel]="searchTerm"
  (ngModelChange)="searchTerm = $event"
>
```

⚠️ **Remember:** You must import FormsModule in your component's `imports` array for `[(ngModel)]` to work.

Use case: Best for form inputs where you want the UI and component data to always stay in sync.

03

Templates & Styles

Angular's powerful template syntax and scoped styling system

Control Flow in Templates

@if — Conditional rendering

```html
@if (isLoggedIn) {
  <p>Welcome back, {{ username }}!</p>
} @else if (isLoading) {
  <p>Loading...</p>
} @else {
  <p>Please log in.</p>
}
```

@for — List rendering

```html
@for (student of students; track student.id) {
  <app-student-card
    [name]="student.name"
    [grade]="student.grade"
  />
} @empty {
  <p>No students found.</p>
}
```

@for variables

Variable	Value
$index	Current loop index (0-based)
$first	true on first item
$last	true on last item
$even	true on even indexes
$odd	true on odd indexes
$count	Total number of items

track is required in `@for` — it tells Angular how to identify items for efficient DOM updates. Use a unique field like `id`.

Template Reference Variables

Referencing DOM elements with #var

```html
<!-- #nameInput is a ref to the input element -->
<input #nameInput type="text"
       placeholder="Enter name">

<!-- Pass the element's value directly -->
<button (click)="greet(nameInput.value)">
  Greet
</button>

<!-- Display it inline -->
<p>You typed: {{ nameInput.value }}</p>
```

```ts
// In the component — no extra property needed!
greet(name: string) {
  alert('Hello, ' + name);
}
```

Why use template references?

- → Read DOM element values without creating a component property
- → Pass values directly to methods
- → Reference child component instances
- → No `[(ngModel)]` needed for simple read cases

Referencing a child component

```html
<app-timer #timerRef></app-timer>
<button (click)="timerRef.start()">
  Start
</button>
```

Component Styles & Encapsulation

```ts
@Component({
  selector: 'app-card',
  template: `<h1>Card Title</h1>`,
  styles: `
    /* Only affects THIS component */
    h1 { color: #dd0031; }
  `,
  encapsulation: ViewEncapsulation.Emulated
})
```

Special CSS selectors

```css
/* Target the component's host element */
:host {
  display: block;
  padding: 16px;
  border: 1px solid #ddd;
}

/* When the host has a class */
:host(.highlighted) {
  border-color: #dd0031;
}

/* Pierce encapsulation — use sparingly! */
:host ::ng-deep .child-class {
  color: red;
}
```

Encapsulation modes

Mode	Behavior
Emulated default	Styles scoped via unique attribute (e.g. `_nghost-abc`)
ShadowDom	True browser Shadow DOM isolation
None	No encapsulation — styles go global. Use very sparingly.

How Emulated works: Angular adds a unique attribute like `_ngcontent-xyz` to every element. Your CSS is then scoped to only match elements with that attribute — simulating Shadow DOM without actually using it.

04

Lifecycle Hooks

Tap into key moments of a component's life

Component Lifecycle

- **`constructor()`** Class instantiated — dependency injection only. Angular hasn't run yet.
- **`ngOnChanges()`** Called when an input property changes. Receives a `SimpleChanges` object.
- **`ngOnInit()`** Called once after first ngOnChanges. ← **Most important hook** — use for initialization logic & data fetching.
- **`ngDoCheck()`** Every change detection run — custom change detection (advanced, rare).
- **`ngAfterViewInit()`** After view initialized — safe to access child component/DOM references.
- **`ngOnDestroy()`** Before component is destroyed. ← **Critical for cleanup** — unsubscribe, clear timers.

> **The big three:** You'll use ngOnInit (setup), ngOnChanges (react to inputs), and ngOnDestroy (cleanup) in virtually every component.

ngOnInit — Initialization

```ts
import { Component, OnInit } from '@angular/core';

@Component({ ... })
export class StudentListComponent
  implements OnInit {

  students: Student[] = [];

  ngOnInit(): void {
    // ✅ Good: initialize data here
    this.students = this.loadStudents();
  }

  private loadStudents(): Student[] {
    return [
      { id: 1, name: 'Alice', grade: 95 },
      { id: 2, name: 'Bob', grade: 82 },
    ];
  }
}
```

constructor vs ngOnInit

constructor	ngOnInit
Dependency injection	Initialize logic
Angular not ready yet	Angular is ready
Inputs not yet set	Inputs are available
No async work	Can call services

Common mistake: Putting initialization logic in the constructor. Inputs (`@Input()`) are NOT set in the constructor — they're set before `ngOnInit`.

Implements: Always declare `implements OnInit` — TypeScript will warn you if you misspell the method name.

ngOnChanges — React to Inputs

```ts
import {
  Component, Input,
  OnChanges, SimpleChanges
} from '@angular/core';

@Component({ ... })
export class StudentCardComponent
  implements OnChanges {

  @Input() grade: number = 0;
  gradeStatus: string = '';

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['grade']) {
      const val = changes['grade'].currentValue;
      const prev = changes['grade'].previousValue;
      console.log(`Grade: ${prev} → ${val}`);

      this.gradeStatus =
        val >= 60 ? 'Passing' : 'Failing';
    }
  }
}
```

SimpleChanges object

```ts
changes['grade'] = {
  currentValue: 95,
  previousValue: 82,
  firstChange: false,
  isFirstChange(): { ... }
}
```

→ currentValue — the new value

→ previousValue — old value (or `undefined`)

→ firstChange — true on first call

When to use: Derive computed values from inputs, log changes, trigger side effects when a specific input changes. Runs *before* ngOnInit on first load.

ngOnDestroy — Cleanup

```ts
import { Component, OnInit, OnDestroy }
  from '@angular/core';

@Component({
  selector: 'app-timer',
  template: `<p>Elapsed: {{ seconds }}s</p>`
})
export class TimerComponent
  implements OnInit, OnDestroy {

  seconds = 0;
  private intervalId: any;

  ngOnInit(): void {
    // Start timer when component appears
    this.intervalId = setInterval(() => {
      this.seconds++;
    }, 1000);
  }

  ngOnDestroy(): void {
    // CRITICAL: cleanup when component is removed
    clearInterval(this.intervalId);
    console.log('Timer cleaned up!');
  }
```

⚠️ Memory Leaks

Without ngOnDestroy, resources keep running even after the component is gone:

→ `setInterval` keeps firing forever

→ Observable subscriptions keep emitting

→ Event listeners keep consuming memory

→ WebSocket connections stay open

Rule: Every resource you *create* in ngOnInit, you must *destroy* in ngOnDestroy. It's like finally blocks in try-catch.

Test it: Use `@if` to toggle the component — watch the console to verify cleanup fires.

Parent ↔ Child Communication

Parent → Child with input()

```ts
// child: student-card.ts
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-student-card',
  template: `
    <h3>{{ name() }}</h3>
    <p>Grade: {{ grade() }}</p>
  `
})
export class StudentCardComponent {
  name = input.required<string>();  // required
  grade = input<number>(0);           // optional
}

// parent template
<app-student-card
  [name]="student.name"
  [grade]="student.grade"
/>
```

Child → Parent with output()

```ts
import { Component, input, output } from
'@angular/core';

@Component({
  selector: 'app-student-card',
  template: `
    <h3>{{ name() }}</h3>
    <button (click)="onDelete()">Delete</button>
  `
})
export class StudentCardComponent {
  name = input.required<string>();
  id = input.required<number>();
  delete = output<number>();

  onDelete() { this.delete.emit(this.id()); }
}

// parent template
<app-student-card
  [name]="s.name"
  (delete)="removeStudent($event)"
/>
```

Core principle: Data flows DOWN via `input()`. Events flow UP via `output()`. This is *unidirectional data flow* — it makes apps predictable and easy to debug.

Live Coding

Multi-Component Task App with Lifecycle Hooks

WHAT WE'LL BUILD

- → Parent component manages task list
- → Child `TaskItemComponent` for each task
- → Pass tasks down with `input()`
- → Emit delete events up with `output()`

LIFECYCLE IN ACTION

- → ngOnInit — log when task appears
- → ngOnChanges — detect task updates
- → ngOnDestroy — log when task is deleted
- → Timer showing time-on-screen per task

Practice Exercises

EXERCISE 1

ProductCardComponent

Create a component that receives a product object via `input()`. Display name, price, and image. Render 5 products using `@for` in AppComponent.

EXERCISE 2

Delete & Like buttons

Add Delete and Like buttons to ProductCard. Use `output()` to emit the product ID. Parent removes the product from the list.

EXERCISE 3

Track likes locally

Track like count locally in the child. Emit a `liked` event to the parent that logs which product was liked and how many times.

EXERCISE 4

ngOnInit & ngOnDestroy

Log when each ProductCard is created and destroyed. Delete products and observe the console. Verify cleanup happens.

EXERCISE 5

Timer Component

Create a component with a timer in `ngOnInit`. Display elapsed seconds. Clear interval in `ngOnDestroy`. Toggle with `@if` to see cleanup.

EXERCISE 6

Category filter

Category selector — clicking shows products for that category. Use `ngOnChanges` in child to detect when the category changes.

Summary

What you learned today

Topic	Key Takeaway
@Component	selector, template, styles, imports
Data Binding	{{ }}, [ ], ( ), [( )]
Templates	@if, @for, template ref vars
Styles	Scoped by default, :host selector
Lifecycle	Init → Changes → Destroy
input()	Data flows down (parent → child)
output()	Events flow up (child → parent)

Best practices

→ Keep components small and focused — one job each

→ Use `ngOnInit` for setup, not constructor

→ Always cleanup in `ngOnDestroy`

→ Data down, events up — always

→ Prefer `input()` / `output()` over decorators

→ Use CLI to generate components consistently

Next lecture: Angular Services & Dependency Injection — how to share logic and data across multiple components without passing everything through inputs.

?

Q & A

Ask anything — we'll debug it together.

How to reach me

Seitbekov Sanzhar

Lecturer

s.seitbekov@kbtu.kz

Office 178

Course chat

Microsoft Teams

Team code: **ksg13tw**

→ Announcements + materials

→ Homework questions

→ Useful links (cheatsheets, docs)